



# Zig

A FAST & FRIENDLY CHEATSHEET

Zig is a **simple, robust, and optimal** systems language — no hidden control flow, no hidden allocations, full power over memory, and a **compile-time** superpower called `comptime`.

0

HIDDEN ALLOCS

1

BUILD SYSTEM

C

ABI NATIVE

∞

COMPTIME

## 1 Hello, Ziggitt!

### Your first program

```
const std = @import("std");

pub fn main() !void {
    const stdout =
    std.io.getStdOut().writer();
    try stdout.print(
        "Hello, {s}!\n",
        .{"Ziggitt"},
    );
}
```

`!void` means "returns void OR an error". `try` propagates errors up.

### Build & run

```
# run a single file
$ zig run hello.zig

# build a debug binary
$ zig build-exe hello.zig

# release modes (pick your trade-off)
$ zig build-exe hello.zig -O ReleaseFast
$ zig build-exe hello.zig -O ReleaseSafe
$ zig build-exe hello.zig -O ReleaseSmall

# run all tests in a file
$ zig test mymodule.zig
```

## 2 What's inside

|   |                         |     |   |                    |     |
|---|-------------------------|-----|---|--------------------|-----|
| 1 | Hello world & toolchain | p.1 | 2 | Variables & types  | p.2 |
| 3 | Control flow & loops    | p.2 | 4 | Functions & errors | p.3 |
| 5 | Structs, enums, unions  | p.3 | 6 | Pointers & slices  | p.4 |

**The Zig motto:** *"In Zig, there are no hidden control flow, no hidden memory allocations, and no preprocessor or macros."* What you read is what runs.

3

## Variables & constants

### const vs var

```
const pi: f64 = 3.14159;           // immutable
var age: u32 = 21;                 // mutable
const name = "Zig";               // type
inferred

// must be initialized – use undefined
// only if you'll write to it before reading
var buf: [64]u8 = undefined;
```

Prefer `const`. The compiler insists if a `var` is never mutated.

### Primitive types

|                                  |                        |
|----------------------------------|------------------------|
| i8 i16 i32<br>i64 i128           | signed integers        |
| u8 u16 u32<br>u64 u128           | unsigned integers      |
| i7 u3 u1024                      | any bit-width allowed! |
| usize /<br>isize                 | pointer-sized          |
| f16 f32 f64<br>f128              | floats                 |
| bool                             | true / false           |
| void ·<br>noreturn ·<br>anyerror | special                |
| type                             | types are values       |

### Arrays · slices · strings

```
const nums = [_]u8{ 1, 2, 3, 4 }; // [4]u8
const first = nums[0];
const len = nums.len;

// slice: pointer + length
const tail: []const u8 = nums[1..];

// strings ARE slices of u8
const hi: []const u8 = "hello";
```

### Optionals — no null surprises

```
var maybe: ?i32 = null;
maybe = 42;

if (maybe) |n| {
    // n is i32 here, never null
    std.debug.print("got {}\n", .{n});
} else {
    std.debug.print("empty\n", .{});
}

const n = maybe orelse 0;           // default
const n2 = maybe.?;                // panic if
null
```

4

## Control flow that's actually fun

### if / else (also expressions)

```
const kind = if (n > 0) "pos"
              else if (n < 0) "neg"
              else "zero";
```

### switch — exhaustive & expressive

```
const msg = switch (status) {
    200          => "OK",
    301, 302     => "redirect",
    400...499    => "client error",
    else        => "other",
};
```

## while — with continue expression

```
var i: u32 = 0;
while (i < 10) : (i += 1) {
    if (i == 3) continue;
    if (i == 7) break;
}

// loop with optional unwrap
while (it.next()) |item| {
    process(item);
}
```

## for — over slices & ranges

```
const xs = [_]u8{ 10, 20, 30 };

for (xs) |x| sum += x;

// with index
for (xs, 0..) |x, i| {
    print("{}: {}\n", .{ i, x });
}

// two slices in lockstep
for (keys, values) |k, v| { ... }
```

## • Labeled blocks & loops · break with a value

```
// blocks are expressions
const answer = blk: {
    var x: i32 = 10;
    x *= 2;
    break :blk x + 2;
};
// answer == 22
```

```
// labeled loop, break with value
const idx = found: for (xs, 0..) |x, i| {
    if (x == target) break :found i;
} else blk: {
    break :blk xs.len; // not found
};
```

## 5 Functions

### Definitions

```
fn add(a: i32, b: i32) i32 {
    return a + b;
}

pub fn greet(name: []const u8) void { ... }

// generic-looking via 'anytype'
fn double(x: anytype) @TypeOf(x) {
    return x * 2;
}
```

pub

fn

return

!T

anytype

### defer · errdefer

```
fn read(path: []const u8) ![]u8 {
    const f = try
        std.fs.cwd().openFile(path, .{});
    defer f.close(); // always runs

    const buf = try alloc.alloc(u8, 4096);
    errdefer alloc.free(buf); // only on
    error

    _ = try f.read(buf);
    return buf;
}
```

Cleanup is local & visible. No try/finally noise.

## 6 Errors are values, not exceptions

### Define · return · propagate

```
const ParseError = error{
    InvalidDigit,
    Overflow,
    Empty,
};

fn parseU8(s: []const u8) ParseError!u8 {
    if (s.len == 0) return error.Empty;
    // ... return parsed value or error
}

const n = try parseU8(s); // propagate
```

### Catch · switch · capture

```
const n = parseU8(s) catch 0; // default

const n2 = parseU8(s) catch |err| switch
    (err) {
        error.Empty      => return error.Bad,
        error.Overflow    => std.math.maxInt(u8),
        error.InvalidDigit => return err,
    };

// in tests, "if errored" check:
// try testing.expectError(error.Empty,
//     parseU8(""));
```

Cheat: `try expr = expr catch |e| return e`. Use `!T` as shorthand for an inferred error set returning `T`.

## 7 Structs, enums & unions

### Struct + methods

```
const Point = struct {
    x: f32,
    y: f32,

    pub fn init(x: f32, y: f32) Point {
        return .{ .x = x, .y = y };
    }

    pub fn len(p: Point) f32 {
        return @sqrt(p.x*p.x + p.y*p.y);
    }
}
```

### Enum (with values + methods)

```
const Color = enum(u8) {
    red   = 0xFF,
    green = 0x0F,
    blue  = 0x0B,

    pub fn isWarm(c: Color) bool {
        return c == .red;
    }
};
```

```
    }  
};  
  
const p = Point.init(3, 4);  
const d = p.len();    // → 5.0
```

```
const c: Color = .red;  
const v = @intFromEnum(c); // 255
```

### ● Tagged union (sum type)

```
const Shape = union(enum) {  
    circle:    f32,  
    square:    f32,  
    rectangle: struct { w: f32, h: f32 },  
};  
  
const s: Shape = .{ .circle = 2.0 };  
  
const area = switch (s) {  
    .circle    => |r| 3.14 * r * r,  
    .square    => |a| a * a,  
    .rectangle => |r| r.w * r.h,  
};
```

### ● Anonymous structs & field init

```
// pass an anonymous struct literal  
configure(.{  
    .timeout_ms = 500,  
    .retries    = 3,  
});  
  
// tuple-style anon (positional)  
const tup = .{ 1, "two", 3.0 };  
const a = tup[0];  
const b = tup[1];
```

8



## Pointers & slices

### The pointer family

\*T

\*const T

[]T

[]const T

[\*]T

[\*c]T

```
var n: i32 = 42;
const p: *i32 = &n;           // single-item ptr
p.* = 100;                     // deref & assign

var arr = [_]i32{ 1, 2, 3, 4 };
const sl: []i32 = arr[1..3];   // slice
const length = sl.len;        // 2
const ptr     = sl.ptr;        // [*]i32
```

arr[0]  
1arr[1]  
2 ← sl[0]arr[2]  
3 ← sl[1]arr[3]  
4

### Mental model

\*T single item, never null

?\*T nullable single-item ptr

[]T slice = ptr + length

[N]T fixed-size array

[\*]T unknown-length, no .len

[\*c]T C-compatible, may be null

.\* / .&amp; deref / address-of

**No null deref by accident:** non-optional pointers are guaranteed non-null. Want maybe-null? Use `?*T`.

9



## Memory · allocators are explicit

### Pick the right allocator for the job

need memory



choose allocator



alloc / create



defer free

```
const std = @import("std");

// 1. General-purpose (debug-friendly, leak-detecting)
var gpa = std.heap.GeneralPurposeAllocator(.{}){};
defer _ = gpa.deinit();
const alloc = gpa.allocator();

// 2. Arena – free everything at once (great for batches)
var arena = std.heap.ArenaAllocator.init(alloc);
defer arena.deinit();
const a = arena.allocator();

// 3. Page allocator – direct OS pages
const p = std.heap.page_allocator;

// 4. Fixed buffer – no syscalls, embedded-friendly
var buf: [4096]u8 = undefined;
var fba = std.heap.FixedBufferAllocator.init(&buf);

// – using one –
const bytes = try alloc.alloc(u8, 256);
defer alloc.free(bytes);

const node = try alloc.create(Node);
defer alloc.destroy(node);
```

### Generics via comptime types

```
fn Stack(comptime T: type) type {
    return struct {
        items: []T,
        len: usize = 0,

        pub fn push(s: *@This(), v: T) void {
            s.items[s.len] = v;
            s.len += 1;
        }
    };
}

var buf: [8]i32 = undefined;
var s = Stack(i32){ .items = &buf };
s.push(42);
```

### comptime — code that runs at compile time

```
// computed at compile time
const table = comptime blk: {
    var t: [256]u8 = undefined;
    for (&t, 0..) |*x, i|
        x.* = @intCast(i * 2);
    break :blk t;
};

// type as parameter
fn printAny(comptime T: type, v: T) void {
    std.debug.print("{any}\n", .{v});
}
```

### Tests are first-class

```
const testing = std.testing;

test "add works" {
    try testing.expectEqual(
        @as(i32, 5),
        add(2, 3),
    );
}

test "errors propagate" {
    try testing.expectError(
        error.Empty,
        parseU8(""),
    );
}

// run with: zig test mod.zig
```

### Builtins you'll actually use

|                      |                        |
|----------------------|------------------------|
| @import              | load a module          |
| @TypeOf              | type of expression     |
| @sizeOf @alignOf     | memory layout          |
| @as                  | explicit type coercion |
| @intCast @floatCast  | numeric casts          |
| @intFromEnum         | enum → int             |
| @ptrCast @alignCast  | pointer casts          |
| @embedFile           | bake file into binary  |
| @panic @compileError | stop the world         |

**Zig in one breath:** small language, big leverage. Read the code = know what runs. Reach for **comptime** to delete duplication, lean on the **std** library, and let the compiler help you free every byte you allocate. **Happy zigging!** ✂